

Sundt-Lejon Technical Manual

Fredrik Sundt & Markus Lejon

July 16, 2026

Contents

1	Introduction	2
1.1	Resources	2
2	Configuration of IsaacSim and IsaacLab	2
2.1	Bokertov	2
2.2	IsaacSim	3
2.3	IsaacLab	4
2.4	Train a RL policy in Isaac Lab	5
3	Spot RL Researcher Kit & NVIDIA Jetson Orin	6
3.1	Jetson configuration	6
3.1.1	Install Ubuntu and JetPack	6
3.1.2	Set up Jetson for Spot	7
3.1.3	Configure the deployment code	9
3.1.4	Changes done to the deployment code	10
3.1.5	Mount Jetson with the mounting kit	11
4	Run the policy	11
4.1	Exporting the policy	11
4.2	Running the policy	12

1 Introduction

This document describes how we configured our systems for training and deploying reinforcement learning (RL) policies for Boston Dynamic’s Spot robot with an attached manipulator arm. Or as we call it: Spot with arm.

Section 2 explains how to install the simulation and training environment, Isaac Sim and Isaac Lab. It also points you to NVIDIA’s documentation to get started with training policies, and where our code can be found.

Section 3 goes through the configuration needed to deploy a trained RL policy on Spot with the Spot RL Researcher Kit.

Section 4 explains how to deploy and run a trained RL policy on Spot.

1.1 Resources

Links to good resources used during our thesis and this manual.

- Isaac Sim Documentation:
<https://docs.isaacsim.omniverse.nvidia.com/latest/index.html>
- Isaac Lab Documentation:
<https://isaac-sim.github.io/IsaacLab/main/index.html>
- Isaac Lab GitHub:
<https://github.com/isaac-sim/IsaacLab>
- Deployment Code GitHub:
<https://github.com/boston-dynamics/spot-rl-example>
- Spot RL Researcher Kit User Guide:
<https://support.bostondynamics.com/s/article/Spot-RL-Researcher-Kit-User-Guide-49965>

2 Configuration of IsaacSim and IsaacLab

2.1 Bokertov

All simulations and training in IsaacSim and IsaacLab is executed on LTH’s computing machine Bokertov.

There are two methods to access Bokertov:

- SSH for command-line interaction via terminal.
- SSH using ThinLinc for graphical remote desktop environment.

Contact Markus Klang to set up an account on Bokertov and to get more information regarding the server.

2.2 IsaacSim

This section explains how to set up IsaacSim using Docker containers. This was done on the server Bokertov, but should be applicable on any device. The section is based on NVIDIA's documentation for installing Isaac Sim (version 5.1.0), which is recommended to read through. https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/install_container.html.

The NVIDIA documentation contains three parts, **Container setup**, **Container deployment**, and **Container deployment with GUI**. The first section explains how to install Docker and the NVIDIA Container Toolkit, this was already done inside Bokertov and therefore skipped in this guide. The second and third part explains how to install and run the IsaacSim container in headless mode and with GUI respectively. We used the GUI deployment method, which the following part will explain how to set up.

1. Run the following command to confirm your GPU driver version:

```
nvidia-smi
```

2. Pull the IsaacSim container:

```
docker pull nvcr.io/nvidia/isaac-sim:5.1.0
```

3. Create the cached volume mounts on host. The guide creates them at the root, we instead created them in the current directory.

```
mkdir -p /docker/isaac-sim/cache/main/ov
mkdir -p /docker/isaac-sim/cache/main/warp
mkdir -p /docker/isaac-sim/cache/computecache
mkdir -p /docker/isaac-sim/config
mkdir -p /docker/isaac-sim/data/documents
mkdir -p /docker/isaac-sim/data/Kit
mkdir -p /docker/isaac-sim/logs
```

```
mkdir -p /docker/isaac-sim/pkg
sudo chown -R 1234:1234 /docker/isaac-sim
```

Note: In order for the container to get access to the cached volumes their permissions must be changed. This is because the user and group of the IsaacSim container is 1234 instead of root. To change the permission sudo access is needed.

4. Run the Isaac Sim container with an interactive Bash session:

```
xhost +local:
docker run --name isaac-sim --entrypoint bash -it --gpus all \
-e "ACCEPT_EULA=Y" --rm --network=host \
-e "PRIVACY_CONSENT=Y" \
-v $HOME/.Xauthority:/isaac-sim/.Xauthority \
-e DISPLAY \
-v /docker/isaac-sim/cache/main:/isaac-sim/.cache:rw \
-v /docker/isaac-sim/cache/computecache:/isaac-sim/.nv/ComputeCache:rw \
-v /docker/isaac-sim/logs:/isaac-sim/.nvidia-omniverse/logs:rw \
-v /docker/isaac-sim/config:/isaac-sim/.nvidia-omniverse/config:rw \
-v /docker/isaac-sim/data:/isaac-sim/.local/share/ov/data:rw \
-v /docker/isaac-sim/pkg:/isaac-sim/.local/share/ov/pkg:rw \
-u 1234:1234 \
nvcr.io/nvidia/isaac-sim:5.1.0
```

5. Check if the system is compatible with Isaac Sim:

```
./isaac-sim.compatibility_check.sh
```

6. Start IsaacSim with GUI, run the following command:

```
./runapp.sh
```

2.3 IsaacLab

This section explains how to set up IsaacLab using Docker containers. This was done on the server Bokertov, but should be applicable on any device. The section is based on NVIDIA's IsaacLab documentation Container deployment: <https://isaac-sim.github.io/IsaacLab/main/source/deployment/index.html> and its subsection Docker guide: <https://isaac-sim.github.io/IsaacLab/main/source/deployment/docker/index.html>

[hub.io/IsaacLab/main/source/deployment/docker.html](https://github.com/isaac-sim/IsaacLab/blob/main/source/deployment/docker.html), which are recommended to read through.

Start by cloning the IsaacLab repository:

```
git clone git@github.com:isaac-sim/IsaacLab.git
```

To thereafter launch the docker container their script *container.py* is used.

To start the container run the command:

```
./docker/container.py start
```

After starting, enter the container with the following command:

```
./docker/container.py enter
```

To stop the container after exiting, run this command:

```
./docker/container.py stop
```

To copy files between the container and the host machine use the `docker cp` command. This is useful for exporting trained RL policies or importing models and code.

```
# Copy file from container to local path
docker cp CONTAINER:[SRC_PATH] [DEST_PATH]
```

```
# Copy files from local path to container
docker cp [SRC_PATH] CONTAINER:[DEST_PATH]
```

```
# ex copy the file some_file from /workspace/isaacclab/ to the current directory
docker cp isaac-lab-base:/workspace/isaacclab/some_file .
```

```
# ex copy the local file some_file into the container dir /workspace/isaacclab/
docker cp ./some_file isaac-lab-base:/workspace/isaacclab/
```

2.4 Train a RL policy in Isaac Lab

To get started with learning how to train a RL policy in Isaac Lab we recommend to read through NVIDIA's documentation. It includes tutorials which are a good way to learn how Isaac Lab works before jumping into your own

project. <https://isaac-sim.github.io/IsaacLab/main/source/setup/quickstart.html>

The training environments, simulation scripts, and the asset of Spot with arm used in our thesis is located in our GitHub repository inside the directory *isaacsim_data*. <https://github.com/DevMarkusLejon/RLonSpot>

3 Spot RL Researcher Kit & NVIDIA Jetson Orin

To deploy RL policies on Spot we used the RL Researcher Kit in combination with the NVIDIA Jetson AGX Orin. The following section goes through how to configure these.

This RL kit contains the following parts:

1. Joint-level API license
2. Hardware mounting kit - used to mount Jetson on spot
3. Deployment code

This section will go through the steps we followed to configure the kit, based on Boston Dynamics guide. <https://support.bostondynamics.com/s/article/Spot-RL-Researcher-Kit-User-Guide-49965>

3.1 Jetson configuration

3.1.1 Install Ubuntu and JetPack

Follow NVIDIA's guide, <https://developer.nvidia.com/embedded/learn/get-started-jetson-agx-orin-devkit>, to set up the operating system and their software package for AI capabilities. The configuration can either be done with a monitor attached or through headless. Either way, setting up the configuration for headless is recommended for debugging once Jetson is attached on Spot.

3.1.2 Set up Jetson for Spot

Follow NVIDIA's closing the sim to real gap, <https://developer.nvidia.com/blog/closing-the-sim-to-real-gap-training-spot-quadruped-locomotion-with-nvidia-isaac-lab/>, to configure Jetson for Spot specific usage, mainly the section *Deploying the trained RL policy on Spot with Jetson Orin*. It's recommended to do this configuration before mounting it on Spot to make any debugging and installation easier.

Network setup on Jetson

Note: Our GUI configuration of the network from the section **Hardware and network setup on Jetson Orin** in the guide did not work. Therefore we set up the network over headless by configuring the *NetworkManager* in the CLI. The following sections explain how to configure the *NetworkManager* using the CLI for this specific use case.

1. Add the network connection for Spot

```
sudo nmcli connection add \  
type ethernet \  
ifname eth0 \  
id <name> \  
ipv4.method manual \  
ipv4.addresses 192.168.50.5/24 \  
ipv4.gateway 192.168.50.3 \  
ipv6.method ignore \  
connection.autoconnect-priority 5
```

2. Add another connection for access to the internet.

```
sudo nmcli connection add \  
type ethernet \  
ifname eth0 \  
id <name> \  
ipv4.method auto \  
connection.autoconnect-priority 0
```

Note: The auto connect priority should be enough for the OS to manually choose the correct connection depending on if the ethernet port is connected to Spot or to the internet. If not, the command *nmcli connection up "id"*

can be used to swap between the two configurations.

Some useful NetworkManager commands are written below. However, there are many more useful nmcli commands out there.

```
nmcli device - List add NM devices eg. wlan, eth, etc
nmcli connection show - Show all NM connections
nmcli connection show <id> - Show the <id> specific NM connection
nmcli connection up <id> - Activate the connection of <id>
nmcli connection down <id> - Deactivate the connection of <id>
nmcli connection modify <id> <value> - Modify <value> of connection <id>
```

A good test to check if the connection is configured and running properly is the command `ping`. To check if the internet is working you can ping a website, ex `ping google.com`. To check if the connection with Spot is working use: `ping 192.168.80.3`.

Software setup on Jetson

This part is based on section **Software setup on Jetson** in NVIDIA's guide, but with some modifications. It includes configuring and installing the necessary components to run a RL policy. Follow these steps before attaching Jetson to Spot since you need internet connection, i.e don't connect Jetson's ethernet port to Spot.

1. From Jetson's home directory run:

```
mkdir spot-rl-deployment && cd spot-rl-deployment && mkdir models
git clone https://github.com/boston-dynamics/spot-rl-example.git
cd spot-rl-example && mkdir external && cd external && mkdir spot_python_sdk
```

2. Download Spot Python SDK with the joint-level API and unzip the content into the `spot_python_sdk` folder created in the step before. Or git clone <https://github.com/boston-dynamics/spot-sdk.git> while in the folder.

3. Install the deployment code dependencies: Note that we changed the version of `bosdyn` and also installed `scipy`.

```
cd ~/spot-rl-deployment/spot-rl-example
sudo apt update
sudo apt install python3-pip
cd external/spot_python_sdk/prebuilt
```



```
pip3 install bosdyn_api-5.1.1-py3-none-any.whl
pip3 install bosdyn_core-5.1.1-py3-none-any.whl
pip3 install bosdyn_client-5.1.1-py3-none-any.whl
pip3 install pygame
pip3 install pyPS4Controller
pip3 install spatialmath-python
pip3 install onnxruntime
pip3 install scipy
```

Note: The pyPS4Controller is only needed if running with a PS4 controller.

Note 2: When installing bosdyn we encountered a `grpcio` dependancy issue. This was either resolved by upgrading pip `python3 -m pip install --upgrade pip` or installing wheel `pip3 install wheel` or `setuptools pip3 install setuptools`.

3.1.3 Configure the deployment code

The deployment code is installed during step 1 under section **Software setup on Jetson**, from this GitHub <https://github.com/boston-dynamics/spot-rl-example>. This code did not work for us out of the box. Our updated code can be found on our GitHub.

Add Spot arm

In the deployment code the configuration in multiple scripts needs to be updated to accommodate the extra joints and DOFs that come with the arm payload.

Configure the controller

NVIDIAs guide uses a wireless PS4 controller, we used a wired controller instead which the SDL did not automatically detect. To fix this we had to manually point the program to the device.

1. Determine the device name of the controller by listing the input devices:

```
ls /dev/input/
```

This directory contains device nodes for all input devices (e.g., `eventX`, `jsX`). In our case, the controller's device name was `js0`.

2. Configure the controller device in the code by setting the appropriate environment variable:

```
import os
os.environ["SDL_JOYSTICK_DEVICE"] = "/dev/input/js0"
```

By default, SDL detects input devices via the `eventX` interface. However, the controller used in this setup was exposed as `js0`, which SDL did not automatically select. Therefore, the device had to be specified manually using the `SDL_JOYSTICK_DEVICE` environment variable.

Add the code at the beginning of these two scripts:

```
~/spot-rl-deployment/spot-rl example/python/hid/gamepad.py
~/spot-rl-deployment/spot-rl-example/python/utils/test_controller.py
```

3.1.4 Changes done to the deployment code

The base code provided by Boston Dynamics is written for the use case of base Spot. This section will briefly mention the changes we have made to the code to accommodate our needs.

- Modified the DOF of the code to match the added DOF from the mounted arm. Automatically detects the DOF based on the policy observation length (only works for our specific observation spaces).
- Modified the script `mock_spot.py` to `init` in a wanted joint configuration and to update the positions at each step.
- Modified the script `gamepad.py` for more capabilities: `triangle` to start/stop the rl policy controlling Spot, `r1` and `l1` to increase/decrease the action scale.
- Modified `onnx_command_generator.py` for added logging capabilities and modified the functions handling observations.
- Modified `onnx_command_generator.py` by adding new proto functions for fixed joint positions.

Depending on your specific use case you might have to change and update the deployment code.

3.1.5 Mount Jetson with the mounting kit

Follow Boston Dynamics guide for mounting Jetson using the hardware mounting kit as well as configuring the payload and the network. <https://support.bostondynamics.com/s/article/Assemble-the-Spot-RL-Researcher-Kit-Hardware-49920>.

4 Run the policy

This subsection will explain how to copy the trained policy from the training computer into Jetson and how to run the policy on Spot.

4.1 Exporting the policy

Exporting policies into Jetson will probably be done once the computer is mounted on Spot. Therefore, this section assumes that you are connected to Spot's WIFI and that you are able to ssh into Jetson.

1. Export the `env.yaml` and `policy.onnx` file from Isaac Lab into its own folder. Note: the `env.yaml` is in the *params* folder and `policy.onnx` file is in the *exported* folder of the training log directory.
2. Copy the folder to Jetson using SSH.

```
scp -P 20022 -r /path/to/folder/* spot@192.168.80.3:<path_to_copy_files>
```

3. SSH into Jetson.

```
ssh -p 20022 spot@192.168.80.3
```

4. Convert the `env.yaml` file to `env_cfg.json` file:

```
cd ~/spot-rl-deployment/spot-rl-example/python/utils/  
python env_convert.py  
#input the path to the .yaml file e.g ~/env.yaml  
#The file outputs a env_cfg.json file in the same directory as the .yaml file
```

5. Move `env_cfg.json` and the trained model `policy.onnx` file into the models folder:

```
mv env_cfg.json policy.onnx ~/spot-rl-deployment/models
```

4.2 Running the policy

1. SSH into Spot:

```
ssh <jetson_username>@<spot_ip> -p 20022
```

e.g

```
ssh spot@192.168.50.3 -p 20022
```

2. Connect the gamepad controller to Spot's USB port.

3. Run the RL policy:

```
cd ~/spot-rl-deployment/spot-rl-example/python  
python spot_rl_demo.py <spot_ip> ~/spot-rl-deployment/models \  
--gamepad-config ./gamepad_config.json
```

Enter Spot's username and password when prompted. Spot will then stand, but the policy will not take control until you press enter. You can now drive the robot with the gamepad. Press enter again to cleanly sit Spot down and exit.

Note: In our code the triangle button on the gamepad is configured to also start/stop the policy part of the program.